

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
BỘ MÔN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH
ASSIGNMENT 06

MẠNG NƠ-RON HỒI TIẾP (RNN)

Giảng viên hướng dẫn	: TRẦN ĐÌNH QUẾ
Họ và tên sinh viên	: NGUYỄN NGỌC HOÀNG NAM
Mã sinh viên	: B23DCCN585
Lớp	: D23CQCN11-B
Nhóm	: 05

Hà Nội - 2026

TÓM TẮT VÀ PHẠM VI

Báo cáo nghiên cứu mạng nơ-ron hồi tiếp cơ bản (RNN) dưới góc nhìn hàm và chuỗi thời gian. Phép truy hồi, trạng thái ẩn, chia sẻ trọng số, lan truyền ngược qua thời gian và xử lý chuỗi có độ dài khác nhau được giải thích bằng công thức, ví dụ NumPy và hai cách triển khai PyTorch/Keras.

Hai bộ dữ liệu gồm Retailrocket với 2.756.101 sự kiện hành vi và S&P 500 với 619.040 bản ghi ngày của 505 mã cổ phiếu. Nhiệm vụ tương ứng là dự đoán loại hành vi kế tiếp trong phiên và dự đoán log return của phiên giao dịch quan sát tiếp theo. Tập chia theo thời gian và chuẩn hóa dựa trên train tạo cơ sở đánh giá theo hướng quá khứ đến tương lai.

Bốn lượt RNN được kiểm tra từ tập dự đoán, đối chiếu với các phương pháp cơ sở và trực quan hóa theo nhiệm vụ. Việc so sánh hai framework dựa trên cùng dữ liệu, độ dài cửa sổ, số chiều ẩn và khởi tạo; không giả định số học giữa hai backend hoàn toàn giống nhau.

Trên Retailrocket, macro-F1 đạt 0,5821 với PyTorch và 0,5818 với Keras, cao hơn 0,3186 của đối chứng luôn chọn view; tuy nhiên accuracy và log loss không vượt mọi đối chứng. Trên S&P 500, RMSE return của hai RNN khoảng 0,015533, cao hơn 0,015291 của dự đoán trung bình train. Kết quả cho thấy hiệu quả của mô hình phụ thuộc nhiệm vụ và tiêu chí, không được bảo đảm chỉ bởi kiến trúc hồi tiếp.

Từ khóa: RNN; trạng thái ẩn; BPTT; hành vi khách hàng; chuỗi thời gian; dự báo; PyTorch; Keras.

Nguồn và tài nguyên: https://github.com/namnguyen05-vn/PTHTTM_Assignment-06. Trợ lý AI hỗ trợ triển khai, kiểm tra và biên soạn; các thống kê dữ liệu và kết quả kiểm tra được sinh từ tập thực tế.

MỤC LỤC (1/2)

CHƯƠNG 1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU ...	4
1.2. Đơn vị dự đoán và giới hạn quan sát ...	5
CHƯƠNG 2. DỮ LIỆU CHUỖI VÀ HÀM HỒI TIẾP ...	6
2.2. Nơ-ron, phép affine và tanh ...	7
2.3. Trạng thái ẩn và phép truy hồi ...	8
2.4. RNN như hợp thành hàm theo thời gian ...	9
2.5. Ví dụ số của chuỗi ba bước ...	10
2.6. Many-to-one và đầu dự đoán ...	11
2.7. Padding, masking và độ dài thật ...	12
2.8. Cross-entropy và trọng số lớp ...	13
2.9. Hồi quy log return và sai số ...	14
2.10. Lan truyền ngược qua thời gian ...	15
2.11. Tiêu biến, bùng nổ và clipping ...	16
2.12. Kiểm tra đạo hàm bằng sai phân hữu hạn ...	17
2.13. Số tham số và chi phí tính toán ...	18
CHƯƠNG 3. NGUỒN VÀ KIỂM TRA DỮ LIỆU ...	19
3.2. Retailrocket: các trường và ngữ nghĩa ...	20
3.3. Retailrocket: làm sạch và dựng phiên ...	21
3.4. Retailrocket: đặc trưng và mất cân bằng ...	22
3.5. Retailrocket: nhịp thời gian và chuyển hành vi ...	23
3.6. S&P 500: cấu trúc và kiểm tra giá ...	24
3.7. S&P 500: biến đổi nhân quả và cửa sổ ...	25
3.8. S&P 500: phân bố và độ phủ ...	26
3.9. Tập chia và thống kê chuẩn hóa ...	27
CHƯƠNG 4. KIẾN TRÚC VÀ HAI FRAMEWORK ...	28
4.2. RNN trong PyTorch ...	29

MỤC LỤC (2/2)

4.3. Một bước cập nhật PyTorch ...	30
4.4. SimpleRNN trong Keras ...	31
4.5. GradientTape và checkpoint Keras ...	32
4.6. Đối chiếu cài đặt và giới hạn tương đương ...	33
CHƯƠNG 5. THIẾT KẾ THỰC NGHIỆM ...	34
5.2. Validation, chọn checkpoint và test ...	35
5.3. Phương pháp đối chứng ...	36
5.4. Dừng, lưu và tiếp tục thực nghiệm ...	37
5.5. Hồ sơ kiểm chứng chương trình ...	38
CHƯƠNG 6. KẾT QUẢ VÀ THẢO LUẬN ...	39
6.2. Đường học và lựa chọn mô hình hành vi ...	40
6.3. Precision, recall và F1 của từng hành vi ...	41
6.4. Ma trận nhầm lẫn và dạng lỗi ...	42
6.5. Khả năng xếp hạng các lớp hiếm ...	43
6.6. Kết quả dự báo log return S&P 500 ...	44
6.7. Diễn biến huấn luyện dự báo cổ phiếu ...	45
6.8. Giá và return dự báo: pytorch ...	46
6.9. Giá và return dự báo: keras ...	47
6.10. Sai số theo phân phối và thời gian ...	48
6.11. Hai framework và chi phí thực nghiệm ...	49
6.12. Kiểm chứng đầu ra và khả năng truy vết ...	50
CHƯƠNG 7. KẾT LUẬN ...	51
GIỚI HẠN VÀ PHẠM VI DIỄN GIẢI ...	52
TÀI LIỆU THAM KHẢO (1/2) ...	53
TÀI LIỆU THAM KHẢO (2/2) ...	54
PHỤ LỤC. THUẬT NGỮ ...	55

CHƯƠNG 1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

Nhiều hệ thống thông minh nhận dữ liệu không chỉ khác nhau về giá trị mà còn khác nhau về thứ tự xuất hiện. Một quyết định dự đoán cần xác định thông tin nào đã có tại thời điểm ra quyết định. Vì vậy, bài toán chuỗi yêu cầu đồng thời mô hình hóa phụ thuộc và tổ chức quy trình đánh giá phù hợp thời gian.

Câu hỏi nghiên cứu	Bằng chứng
RNN biểu diễn lịch sử bằng hàm như thế nào?	Phương trình truy hồi, ví dụ số và BPTT
Hai dataset có cấu trúc chuỗi gì?	Audit, phân bố, ranh giới thực thể và thời gian
PyTorch và Keras triển khai cùng ý tưởng ra sao?	Đồ thị, masking và đối chiếu NumPy
RNN có cải thiện so với dự đoán đơn giản?	Test theo thời gian và các đối chứng
Kết quả có thể kiểm tra và tiếp tục không?	Checkpoint, optimizer, dự đoán và hồ sơ kiểm chứng

Bảng 1. Các câu hỏi và cách khảo sát.

Phạm vi chỉ gồm RNN cơ bản với một lớp hồi tiếp tanh và một đầu tuyến tính. Hai framework là hai cách cài đặt cùng họ mô hình, không phải hai thuật toán học khác nhau. Không đặt mục tiêu tìm cấu hình tối ưu toàn cục hoặc đưa ra chiến lược giao dịch.

1.2. Đơn vị dự đoán và giới hạn quan sát

Với Retailrocket, một mẫu là lịch sử từ một đến hai mươi sự kiện đã quan sát trong cùng phiên và loại sự kiện kế tiếp. Với chứng khoán, một mẫu là ba mươi phiên quan sát trước đó của cùng mã và log return của phiên tiếp theo.

Yếu tố	Retailrocket	S&P 500
Thực thể	Khách truy cập và phiên	Mã cổ phiếu
Trục thứ tự	Timestamp sự kiện	Ngày giao dịch
Đầu vào	Hành vi, khoảng cách, lịch thời gian	Đặc trưng giá/khối lượng quá khứ
Đầu ra	View/cart/transaction	Log return
Giới hạn	Có điều kiện phiên còn sự kiện	Dữ liệu lịch sử và nguồn giá

Bảng 2. Đơn vị phân tích ở hai nhiệm vụ.

Hành vi giao dịch không đồng nhất với số đơn hàng hoặc doanh thu; nhiều dòng có thể thuộc cùng một giao dịch. Dự đoán loại hành vi tiếp theo cũng không tương đương dự đoán khả năng mua của mọi khách truy cập, vì khách không còn sự kiện không có nhãn kế tiếp trong thiết kế.

Đối với chứng khoán, dữ liệu là một panel nhiều chuỗi. Trộn ngẫu nhiên các cửa sổ vào train và test sẽ khiến quan sát tương lai có thể tham gia học trước khi đánh giá quá khứ. Bài giữ ranh giới thời gian toàn cục chung cho mọi mã.

CHƯƠNG 2. DỮ LIỆU CHUỖI VÀ HÀM HỒI TIẾP

Dữ liệu chuỗi là một dãy quan sát có thứ tự. Vị trí của một phần tử mang ý nghĩa bên cạnh giá trị của nó. Hai phiên truy cập cùng có một lần xem và một lần thêm giỏ hàng nhưng khác thứ tự có thể biểu thị các tình huống khác nhau. Tương tự, một chuỗi tăng rồi giảm giá không tương đương với chuỗi giảm rồi tăng.

$$X = (x_1, x_2, \dots, x_T), \quad x_t \in \mathbb{R}^D$$

Ký hiệu	Ý nghĩa
T	Số bước thời gian trong cửa sổ
D	Số đặc trưng ở mỗi bước
H	Số chiều trạng thái ẩn
B	Số mẫu trong minibatch

Bảng 3. Các chiều tensor trong mô hình chuỗi.

Bước thời gian là vị trí quan sát, không mặc định là một khoảng thời gian vật lý cố định. Retailrocket có khoảng cách giữa các sự kiện không đều; chúng khoán được quan sát theo phiên giao dịch. Việc đưa khoảng cách thời gian vào đặc trưng không biến RNN thành mô hình thời gian liên tục, nhưng cung cấp thêm thông tin về nhịp độ hành vi.

Một mẫu huấn luyện trong bài là một cửa sổ lịch sử và một nhãn tương lai. Mã khách hàng hoặc mã cổ phiếu xác định ranh giới chuỗi; không được nối hai thực thể chỉ vì chúng xuất hiện cạnh nhau trong tệp CSV.

2.2. Nơ-ron, phép affine và tanh

Phép affine kết hợp tuyến tính đầu vào rồi cộng độ lệch. Một hàm kích hoạt phi tuyến được áp dụng sau đó để mở rộng lớp hàm mà mô hình có thể biểu diễn. Nếu nhiều lớp chỉ gồm phép affine, hợp thành của chúng vẫn là một phép affine.

$$z = xW + b$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh'(z) = 1 - \tanh^2(z)$$

```
z = np.array([-2., 0., 2.])
activation = np.tanh(z)
derivative = 1 - activation**2
```

Mã 1. Giá trị và đạo hàm của tanh.

Tanh đưa mỗi thành phần trạng thái vào khoảng $(-1, 1)$. Ở vùng gần không, hàm phản ứng tương đối mạnh với thay đổi đầu vào; ở vùng bão hòa, đạo hàm gần không. Giới hạn biên độ trạng thái không đồng nghĩa giới hạn toàn bộ gradient theo tham số, vì đường đạo hàm còn đi qua các ma trận trọng số.

Trong dữ liệu nhiều thang đo, chuẩn hóa đầu vào giúp tránh một số biến số lớn áp đảo ngay từ phép affine đầu tiên. Thống kê chuẩn hóa được ước lượng trên giai đoạn train; dùng cả test để ước lượng sẽ đưa thông tin phân phối tương lai vào quá trình xây dựng mô hình.

2.3. Trạng thái ẩn và phép truy hồi

RNN cơ bản duy trì một vector trạng thái ẩn để biểu diễn lịch sử đã quan sát. Trạng thái mới phụ thuộc đồng thời đầu vào hiện tại và trạng thái trước. Đây là cơ chế hồi tiếp: đầu ra trung gian ở một bước trở thành đầu vào của bước kế tiếp [1].

$$h_0 = 0, \quad h_t = \tanh(x_t W_x + h_{t-1} W_h + b_h)$$

$$z = h_T W_y + b_y$$

Tham số	Kích thước	Vai trò
W_x	$D \times H$	Biến đổi quan sát hiện tại
W_h	$H \times H$	Biến đổi trạng thái trước
b_h	H	Độ lệch của trạng thái
W_y, b_y	$H \times C; C$	Ánh xạ trạng thái sang đầu ra

Bảng 4. Các tham số theo quy ước vector hàng.

Trạng thái ẩn không phải bản sao toàn bộ dữ liệu quá khứ và không có nghĩa mỗi chiều tương ứng một khái niệm quan sát được. Nó là biểu diễn số học được học để giảm hàm mất mát. Với $H=32$, toàn bộ lịch sử trong cửa sổ được tổng hợp thành 32 giá trị trước đầu dự đoán.

Trong bài, h_0 được đặt bằng không ở mỗi cửa sổ. Trạng thái không được truyền tùy tiện từ minibatch trước sang minibatch sau, vì các mẫu trong batch có thể thuộc những thực thể khác nhau.

2.4. RNN như hợp thành hàm theo thời gian

$$h_T = f_{x_T} \circ f_{x_{T-1}} \circ \dots \circ f_{x_1}(h_0)$$

Với một quan sát x cố định, f_x là hàm biến đổi trạng thái. RNN áp dụng nhiều lần cùng quy tắc tham số hóa trên các quan sát khác nhau. Đồ thị khi được trải theo thời gian có nhiều bước, nhưng các bước dùng chung W_x , W_h và b_h .

```
def recurrent_step(x, h, wx, wh, bias):
    return np.tanh(x @ wx + h @ wh + bias)

h = np.zeros(hidden_size)
for x_t in sequence:
    h = recurrent_step(x_t, h, wx, wh, bias)
```

Mã 2. Vòng lặp thể hiện phép hợp thành theo thời gian.

Chia sẻ trọng số giữ số tham số độc lập với độ dài T . Tuy nhiên, chi phí tính toán vẫn tăng theo T vì trạng thái tại bước sau cần trạng thái của bước trước. Khác với một phép tổng các quan sát, thứ tự thực hiện các hàm nói chung ảnh hưởng kết quả.

Trải đồ thị là cách diễn giải cùng một mô hình, không tạo ra T bộ trọng số độc lập. Nếu mỗi vị trí có tham số riêng, mô hình sẽ có giả định và khả năng xử lý độ dài khác hẳn RNN được sử dụng ở đây.

2.5. Ví dụ số của chuỗi ba bước

```
x = np.array([[1., 0.], [0., 1.], [1., 1.]])
wx = np.array([[.3, -.2], [.1, .4]])
wh = np.eye(2) * .5
bias = np.zeros(2)
h = np.zeros(2)
states = [h.copy()]
for x_t in x:
    h = np.tanh(x_t @ wx + h @ wh + bias)
    states.append(h.copy())
```

Mã 3. Ví dụ hai chiều để quan sát trạng thái từng bước.

Bước	h_1	h_2
0	0,0000	0,0000
1	0,2913	-0,1974
2	0,2408	0,2925
3	0,4780	0,3331

Bảng 5. Trạng thái tính trực tiếp từ ví dụ NumPy.

Ở bước thứ hai, đầu vào chỉ kích hoạt đặc trưng thứ hai nhưng trạng thái vẫn chịu ảnh hưởng của bước thứ nhất qua $h_1 W_h$. Khi đảo thứ tự ba vector đầu vào, phép cộng trạng thái xảy ra ở các thời điểm khác nhau trước tanh, nên trạng thái cuối thường thay đổi.

Ví dụ này không chứa bước học: trọng số được ấn định để làm rõ phép tính. Trong thực nghiệm, gradient điều chỉnh trọng số dựa trên sai khác giữa dự đoán và nhãn. Phân biệt phép suy luận với quá trình học giúp tránh đồng nhất một ví dụ số với một mô hình đã được huấn luyện.

2.6. Many-to-one và đầu dự đoán

Cả hai nhiệm vụ dùng cấu trúc many-to-one: nhiều bước lịch sử tạo một đầu ra. Retailrocket có ba logits cho ba hành vi; chứng khoán có một đầu ra thực cho log return của phiên kế tiếp.

$$p_c = \frac{\exp(z_c)}{\sum_j \exp(z_j)}$$

$$\hat{r}_{t+1} = s_r z, \quad \hat{P}_{t+1} = P_t \exp(\hat{r}_{t+1})$$

Khía cạnh	Hành vi	Chứng khoán
Đầu ra	3 logits	1 giá trị thực
Biến đổi	Softmax	Nhân độ lệch chuẩn train
Quyết định	Argmax	Dự báo log return
Diễn giải thêm	Xác suất từng hành vi	Giá đóng cửa hàm ý

Bảng 6. Hai đầu dự đoán dùng chung lõi hồi tiếp.

Logits là điểm số chưa chuẩn hóa, không phải xác suất. Softmax chỉ áp dụng cho bài phân loại. Đầu hồi quy không dùng softmax vì tổng bằng một và miền giá trị bị ràng buộc sẽ không phù hợp với lợi suất âm hoặc dương.

Dự báo giá được xây dựng từ giá đã biết ở cuối cửa sổ và return dự báo. Khi return dự báo bằng không, giá dự báo bằng giá gần nhất. Do đó đồ thị giá có thể trông gần đúng ngay cả khi mô hình chưa vượt được đối chứng đơn giản.

2.7. Padding, masking và độ dài thật

Các phiên khách hàng có độ dài khác nhau. Để gom chúng thành tensor $B \times T \times D$, chương trình đặt chuỗi thật ở đầu và thêm số không vào phần còn lại. Lengths lưu số bước thật; phần đệm được bỏ qua bằng masking trong Keras và packed sequence trong PyTorch.

```
mask = np.arange(max_length)[None, :] < lengths[:, None]
x[~mask] = 0.0
# Keras receives mask; PyTorch receives packed lengths.
```

Mã 4. Mặt nạ phần đệm theo độ dài thật.

Số không sau chuẩn hóa cũng có thể là một giá trị dữ liệu hợp lệ. Vì thế mask được xây dựng từ lengths, không suy ra bằng điều kiện mọi đặc trưng bằng không. Điều này tách rõ dữ liệu thật khỏi ô trống phục vụ tính toán.

Sai sót	Hệ quả
Lấy trạng thái sau toàn bộ padding	Mô hình tiếp tục biến đổi lịch sử bằng bước giả
Che theo giá trị bằng 0	Có thể bỏ nhầm một bước thật
Đệm trái nhưng xử lý như đệm phải	Độ dài và vị trí quan sát không khớp

Bảng 7. Các lỗi thường gặp khi xử lý chuỗi ngắn.

Kiểm tra độc lập thay phần đệm từ 0 thành 10.000 và yêu cầu dự đoán giữ nguyên. Bài kiểm tra này đánh giá hành vi của mô hình, bổ sung cho việc kiểm tra hình dạng tensor.

2.8. Cross-entropy và trọng số lớp

$$\ell_i = -w_{y_i} \log p_{i, y_i}$$

$$L_B = \frac{1}{|B|} \sum_{i \in B} \ell_i$$

Trong Retailrocket, hành vi xem chiếm phần lớn dữ liệu. Cross-entropy có trọng số tăng đóng góp của lớp hiếm vào gradient. Trọng số được tính từ nhãn train theo căn bậc hai của nghịch đảo tần suất, rồi chuẩn hóa để trung bình theo mẫu bằng một.

```
counts = np.bincount(y_train, minlength=3)
weight = np.sqrt(counts.sum() / (3*np.maximum(counts, 1)))
weight /= np.dot(counts, weight) / counts.sum()
loss = (per_sample_cross_entropy * weight[y]).mean()
```

Mã 5. Trọng số chỉ dựa trên phân bố lớp train.

Trọng số không tạo thêm thông tin cho lớp hiếm và không bảo đảm recall tăng trong mọi lượt. Nó thay đổi mục tiêu tối ưu. Chỉ số validation và test được tính trên phân bố quan sát gốc; không nhân trọng số lại vào accuracy hoặc confusion matrix.

Hai framework đều lấy trung bình loss đã nhân trọng số theo số mẫu trong batch. Quy ước này được ghi rõ vì một số API loss có trọng số có thể dùng mẫu số khác, dẫn đến hai chương trình trông tương tự nhưng cập nhật gradient không tương đương.

2.9. Hồi quy log return và sai số

$$r_{t+1} = \log(P_{t+1}/P_t)$$

$$L_B = \frac{1}{|B|} \sum_i (z_i - r_i/s_r)^2$$

Giá tuyệt đối của các cổ phiếu khác nhau có thang đo khác nhau. Log return biểu diễn thay đổi tương đối và thuận tiện để dùng một mô hình chung cho nhiều mã. Target được chia cho độ lệch chuẩn return của train để giữ thang loss phù hợp; không trừ trung bình target.

Chỉ số	Ý nghĩa
MAE	Trung bình độ lớn sai số return
RMSE	Căn trung bình bình phương sai số; nhạy với sai số lớn
Directional accuracy	Tỷ lệ dấu dự báo trùng dấu return thật
Price MAE/RMSE	Sai số sau quy đổi về giá; nhạy thang giá

Bảng 8. Các chỉ số hồi quy và giới hạn đọc kết quả.

Return đúng bằng không được xem là trạng thái riêng trong phép so dấu. Đối chứng zero-return vì thế không tự có directional accuracy 50%. Chỉ số chính để so với đối chứng là sai số return; accuracy theo hướng chỉ bổ sung thông tin.

Các biến động rất lớn vẫn được giữ trong tập đánh giá và thống kê riêng. Loại chúng chỉ vì mô hình dự báo kém sẽ làm thay đổi bài toán sau khi đã quan sát kết quả.

2.10. Lan truyền ngược qua thời gian

Backpropagation through time (BPTT) áp dụng quy tắc dây chuyền lên đồ thị đã trải. Tại bước cuối, gradient của loss theo trạng thái được truyền ngược qua tanh và qua ma trận hồi tiếp. Vì tham số dùng chung, gradient tham số cộng đóng góp từ mọi bước.

$$\delta_t = \frac{\partial L}{\partial h_t} \odot (1 - h_t^2)$$

$$\nabla_{W_x} L = \sum_t x_t^\top \delta_t, \quad \nabla_{W_h} L = \sum_t h_{t-1}^\top \delta_t$$

```
dh = states[-1] - target
for t in range(len(x)-1, -1, -1):
    dz = dh * (1 - states[t+1]**2)
    dwx += np.outer(x[t], dz)
    dwh += np.outer(states[t], dz)
    db += dz
    dh = dz @ wh.T
```

Mã 6. BPTT cho loss bình phương tại trạng thái cuối của ví dụ NumPy.

Đoạn minh họa dùng loss ở trạng thái cuối để thấy rõ đường đạo hàm; huấn luyện chính có thêm đầu tuyến tính và loss phân loại hoặc hồi quy. Autograd của PyTorch và GradientTape của TensorFlow tính đạo hàm cho đồ thị đầy đủ.

Cửa sổ hữu hạn giới hạn đường BPTT ở 20 hoặc 30 bước trong thiết kế này. Bài không truyền trạng thái giữa các cửa sổ, nên không phải thuật toán học trực tuyến duy trì bộ nhớ xuyên toàn bộ lịch sử.

2.11. Tiêu biến, bùng nổ và clipping

Gradient đi qua nhiều phép nhân ma trận và đạo hàm kích hoạt. Nếu độ lớn bị co lặp lại, tác động của quan sát xa có thể trở nên rất nhỏ; nếu bị khuếch đại, cập nhật có thể mất ổn định. Hai hiện tượng này là khó khăn đặc trưng của RNN [2].

$$g_{\text{clip}} = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right)$$

```
torch.nn.utils.clip_grad_norm_(parameters, max_norm=1.0)
# Equivalent global-norm clipping in TensorFlow:
clipped, norm = tf.clip_by_global_norm(gradients, 1.0)
```

Mã 7. Giới hạn chuẩn toàn cục trước cập nhật Adam.

Với ví dụ vô hướng, 0,5 mũ 20 xấp xỉ $9,54 \times 10^{-7}$, còn 1,5 mũ 20 xấp xỉ 3.325. Ví dụ chỉ minh họa phép nhân lặp; gradient RNN thực tế phụ thuộc trạng thái, hướng vector và các ma trận, không được xác định bằng một hệ số duy nhất.

Clipping hạn chế bước cập nhật khi gradient quá lớn, nhưng không khôi phục gradient đã tiêu biến. Khởi tạo ma trận hồi tiếp trực giao và cửa sổ vừa phải hỗ trợ ổn định ở giai đoạn đầu; chúng không chứng minh mô hình nhớ được mọi phụ thuộc dài hạn.

2.12. Kiểm tra đạo hàm bằng sai phân hữu hạn

$$\frac{\partial L}{\partial \theta_j} \approx \frac{L(\theta + \epsilon e_j) - L(\theta - \epsilon e_j)}{2\epsilon}$$

Sai phân trung tâm đánh giá độ thay đổi của loss khi một tham số được tăng và giảm một lượng nhỏ. Phương pháp chậm khi có nhiều tham số nhưng hữu ích cho một mạng NumPy nhỏ, nơi có thể đổi chiều từng phần tử của gradient.

```
numeric = finite_difference(
    lambda w: last_state_gradient(x, w, wh, bias, target)[0], wx)
np.testing.assert_allclose(analytic, numeric, atol=1e-7, rtol=1e-6)
```

Mã 8. Đối chiếu gradient giải tích và gradient số.

Epsilon quá lớn tạo sai số xấp xỉ; quá nhỏ có thể làm sai số làm tròn chiếm ưu thế. Ví dụ dùng float64 và epsilon 10^{-6} , còn thực nghiệm framework dùng float32. Kết quả kiểm tra nhỏ xác nhận một đường tính toán cụ thể, không thay thế kiểm chứng dữ liệu và đánh giá mô hình.

Kiểm tra	Đối tượng
Gradient NumPy	W_x, W_h và b_h
Đối chiếu forward	Trạng thái cuối và đầu tuyến tính
Masking	Giá trị padding không đổi đầu ra
Checkpoint	Trạng thái optimizer và trọng số khi tiếp tục

Bảng 9. Các tầng kiểm tra độc lập.

2.13. Số tham số và chi phí tính toán

$$P = DH + H^2 + H + HC + C$$

Dataset	D	H	C	Tham số học
Retailrocket	10	32	3	1475
S&P 500	7	32	1	1313

Bảng 10. Số tham số của kiến trúc RNN sử dụng.

PyTorch RNN cung cấp hai vector bias; Keras SimpleRNN dùng một vector. Bản PyTorch cố định bias hồi tiếp bằng không, chỉ học bias đầu vào. Nhờ vậy hai mô hình có cùng số tham số học và cùng dạng hàm; vẫn có một vector không học trong cấu trúc đối tượng PyTorch.

Số tham số nhỏ không đồng nghĩa chi phí huấn luyện không đáng kể. Hàng trăm nghìn cửa sổ phải đi qua nhiều bước hồi tiếp ở mỗi epoch. Tạo cửa sổ khi lấy batch giảm bộ nhớ lưu trữ, nhưng không loại bỏ phép tính cần thiết cho từng chuỗi.

Chi phí một forward có bậc xấp xỉ $B \times T \times (DH + H^2 + HC)$. Biểu thức chỉ mô tả số phép tính; thời gian còn phụ thuộc framework, masking, truyền dữ liệu và kernel phần cứng.

CHƯƠNG 3. NGUỒN VÀ KIỂM TRA DỮ LIỆU

Dataset	Quy mô tệp gốc	Nguồn
Retailrocket	2,756,101 sự kiện	events.csv
S&P 500	619,040 dòng	all_stocks_5yr.csv

Bảng 11. Các tệp được tải và kiểm tra trực tiếp.

Retailrocket: <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

S&P 500: <https://www.kaggle.com/datasets/camnugent/sandp500>

Trình tải lấy riêng hai tệp cần thiết, lưu SHA256 và kiểm tra tên cột. Các tệp thuộc tính sản phẩm của Retailrocket không được sử dụng vì nhiệm vụ hiện tại dự đoán loại hành vi từ lịch sử tương tác. Quy mô được báo cáo theo tệp thật, không cộng các tệp nguồn không tham gia mô hình.

Dữ liệu raw và các mảng prepared nằm ngoài repository mã nguồn. Các mảng được truy cập bằng memory mapping, giúp tạo batch từ cửa sổ cần thiết mà không lưu trước toàn bộ tensor ba chiều.

Retailrocket công bố giấy phép CC BY-NC-SA 4.0; nguồn S&P 500 ghi CC0. Báo cáo dẫn nguồn dataset; repository cung cấp mã tải và thống kê, không nhân bản toàn bộ dữ liệu gốc.

3.2. Retailrocket: các trường và ngữ nghĩa

Trường	Cách hiểu trong nghiên cứu
timestamp	Thời điểm Unix tính bằng mili giây
visitorid	Định danh khách truy cập đã ẩn danh
event	view, addtocart hoặc transaction
itemid	Định danh sản phẩm; dùng kiểm tra lặp sản phẩm
transactionid	Mã giao dịch, không dùng làm đầu vào

Bảng 12. Từ điển dữ liệu hành vi.

Timestamp được đổi về UTC để quy tắc xử lý nhất quán. Nguồn không xác định múi giờ địa phương của từng người dùng; các đặc trưng giờ và ngày trong tuần vì vậy mô tả nhịp thời gian theo UTC, không được diễn giải trực tiếp thành giờ sinh hoạt địa phương.

Thiếu transactionid ở sự kiện xem hoặc thêm giỏ hàng là phù hợp ngữ nghĩa, không tự động là lỗi cần điền. Dùng sự hiện diện của mã giao dịch để dự báo loại sự kiện cùng dòng sẽ gây lộ nhãn; bài loại cột này khỏi đặc trưng.

Itemid chỉ dùng để xác định sản phẩm ở bước hiện tại có giống bước trước trong phiên hay không. Mô hình không học embedding cho hàng trăm nghìn mã sản phẩm; lựa chọn này tập trung nghiên cứu vào động lực loại hành vi và giữ kích thước RNN nhỏ.

3.3. Retailrocket: làm sạch và dựng phiên

Bước kiểm tra	Số lượng
Dòng gốc	2756101
Trùng hoàn toàn bị loại	460
Dòng trùng thời điểm gây thứ tự mơ hồ bị loại	4769
Sự kiện còn lại	2750872
Phiên sau xử lý	1761509
Phiên một sự kiện	1379077

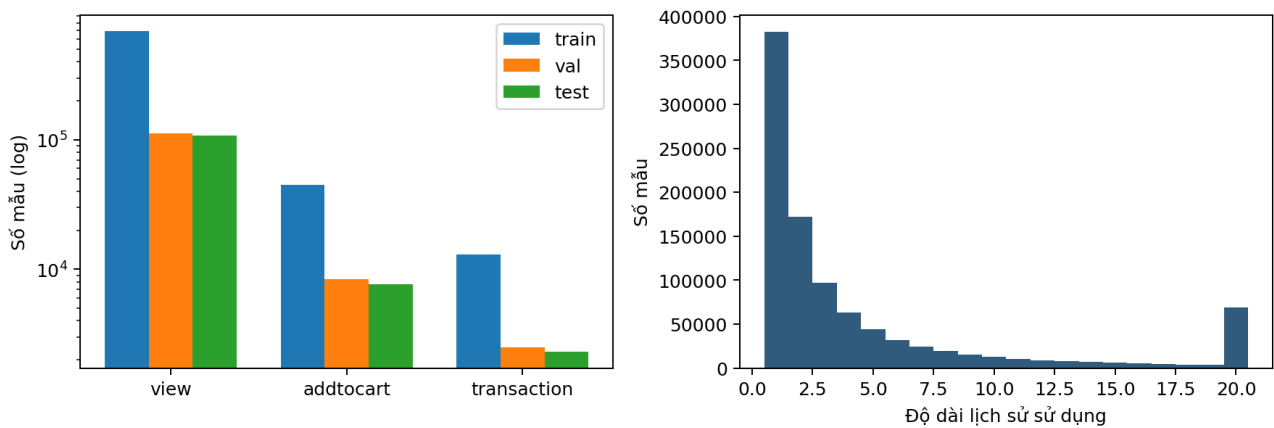
Bảng 13. Audit làm sạch Retailrocket.

Sau khi loại bản ghi trùng hoàn toàn, các dòng cùng visitorid và timestamp nhưng còn nhiều quan sát được loại khỏi chuỗi. Đây là quy tắc bảo thủ để không tự đặt thứ tự giữa các hành động đồng thời. Nó làm mất một phần sự kiện giao dịch nhiều sản phẩm; phân bố sau xử lý không được đồng nhất với toàn bộ lưu lượng ban đầu.

Một phiên mới bắt đầu khi đổi visitorid hoặc khoảng cách từ sự kiện trước vượt ba mươi phút. Ngưỡng này là giả định mô hình hóa của nghiên cứu, không phải nhãn phiên do website cung cấp. Phiên chỉ có một sự kiện không tạo được cặp lịch sử-hành vi tiếp theo.

Các phiên cắt qua ranh giới tập chia bị loại khỏi nhóm mẫu huấn luyện/đánh giá để một phiên không xuất hiện ở nhiều phần. Cùng một khách quay lại ở thời gian khác vẫn có thể xuất hiện ở nhiều tập; bài đánh giá dự báo tương lai của dòng truy cập, không chỉ riêng khách hoàn toàn mới.

3.4. Retailrocket: đặc trưng và mất cân bằng



Hình 1. Phân bố nhãn mục tiêu và độ dài lịch sử sau xử lý.

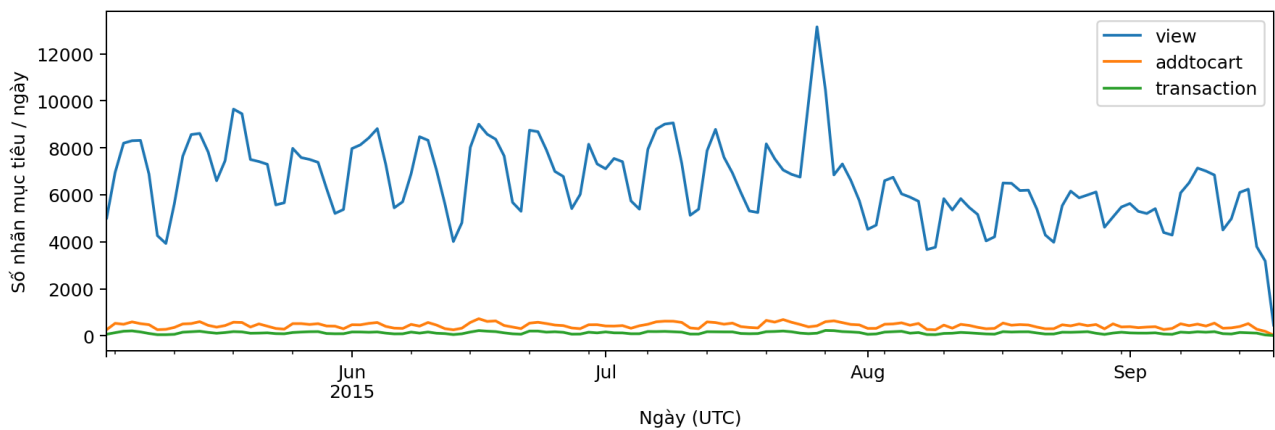
Nhóm đặc trưng	Biểu diễn
Hành vi	Ba biến one-hot
Khoảng cách	$\log(1 + \text{số giây từ sự kiện trước})$
Lặp sản phẩm	Cờ cùng itemid với bước trước
Vị trí trong phiên	$\log(1 + \text{chỉ số vị trí})$
Nhịp thời gian	Sin/cos của giờ và ngày trong tuần

Bảng 14. Mười đặc trưng của một bước hành vi.

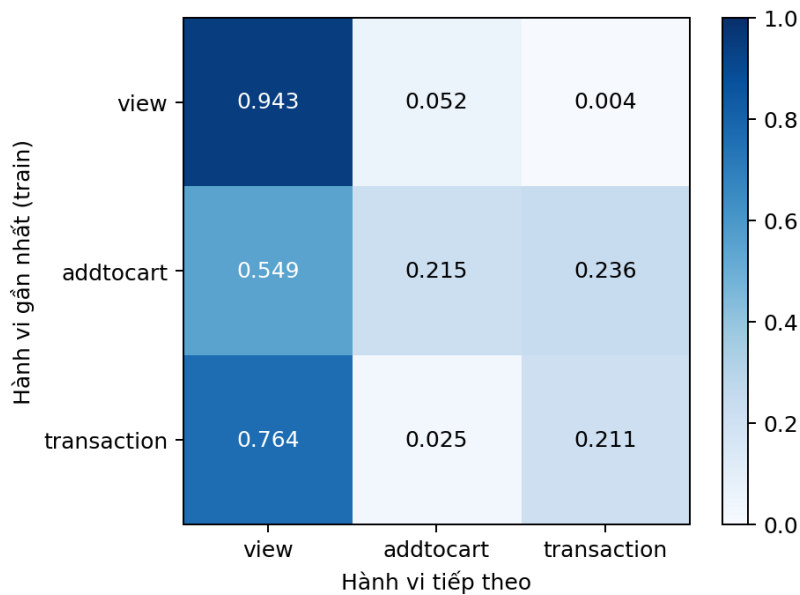
Các biến đều được tính từ sự kiện hiện tại và quá khứ trong cửa sổ. Không dùng thời lượng cuối cùng của phiên, tổng số giao dịch trong tương lai hoặc số bước còn lại làm đặc trưng. Những biến đó chỉ biết sau thời điểm dự báo và có thể tạo kết quả lạc quan giả.

Nhãn view áp đảo nên accuracy cần được đọc cùng macro-F1 và chỉ số từng lớp. Histogram độ dài có mức chặn ở hai mươi bước: giá trị 20 gồm cả lịch sử dài hơn đã được cắt lấy phần gần nhất.

3.5. Retailrocket: nhịp thời gian và chuyển hành vi



Hình 2. Số nhãn mục tiêu theo ngày; đây là các mẫu dự đoán hợp lệ, không phải mọi sự kiện raw.



Hình 3. Xác suất chuyển hành vi ước lượng chỉ từ train, chưa cộng làm trơn.

Ma trận chuyển bậc một chỉ dùng hành vi gần nhất. Nó là một đối chứng cần thiết: nếu RNN không vượt phương pháp này, chưa có bằng chứng rằng lịch sử dài hơn và các đặc trưng bổ sung mang lại lợi ích trong cấu hình đã khảo sát.

3.6. S&P 500: cấu trúc và kiểm tra giá

Trường	Ý nghĩa
date	Ngày quan sát
Name	Mã cổ phiếu
open / close	Giá mở / đóng cửa
high / low	Giá cao / thấp nhất ngày
volume	Khối lượng giao dịch

Bảng 15. Từ điển dữ liệu giá và khối lượng.

Kiểm tra	Số lượng
Dòng gốc	619040
Dòng không hợp lệ bị loại	23
Xung đột mã/ngày bị loại	0
Dòng còn lại	619017
Mã cổ phiếu	505

Bảng 16. Audit dữ liệu S&P 500.

Giá phải hữu hạn và dương, khối lượng không âm; high và low phải bao quanh open và close. Các đoạn có khoảng cách hơn mười ngày lịch được tách để cửa sổ không nối qua khoảng thiếu dài. Một vài ngày nghỉ không được điền thành giao dịch giả.

Thẻ Kaggle không xác lập đầy đủ chính sách điều chỉnh corporate action theo thông tin có sẵn từng thời điểm. Bài ghi nhận giới hạn này, cùng nguy cơ thiên lệch do thành phần chỉ số và các mã còn hiện diện trong bộ dữ liệu.

3.7. S&P 500: biến đổi nhân quả và cửa sổ

$$r_t = \log(C_t/C_{t-1}), \quad q_t = \log(C_t/O_t)$$

$$a_t = \log(H_t/L_t), \quad g_t = \log(O_t/C_{t-1})$$

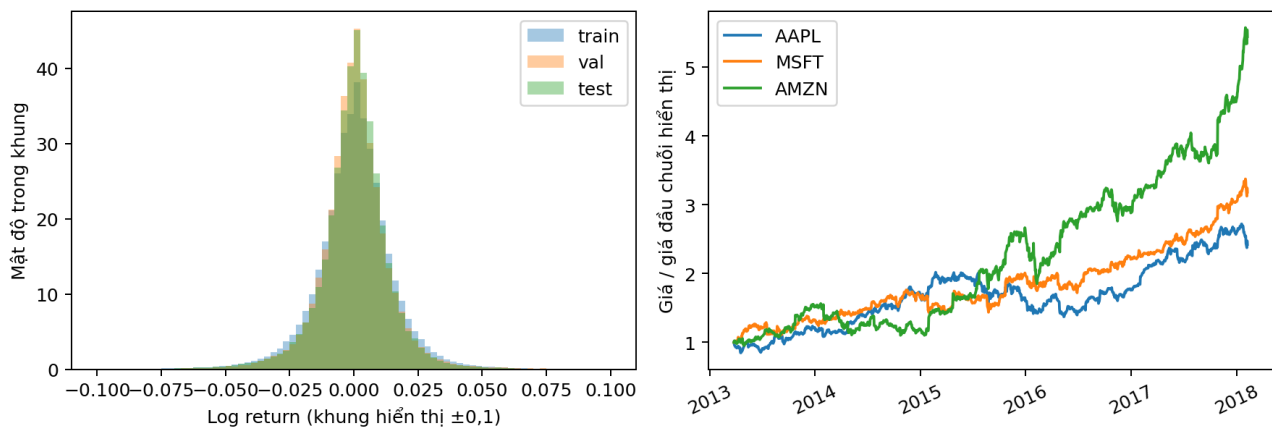
Đặc trưng	Thông tin sử dụng
Log return đóng cửa	Close hiện tại và close trước
Return trong ngày	Close / open
Biên độ trong ngày	High / low
Gap mở cửa	Open hiện tại / close trước
Đổi log volume	Chênh lệch $\log(1 + \text{volume})$
Ngày trong tuần	Hai thành phần sin/cos

Bảng 17. Bảy đặc trưng của một bước chứng khoán.

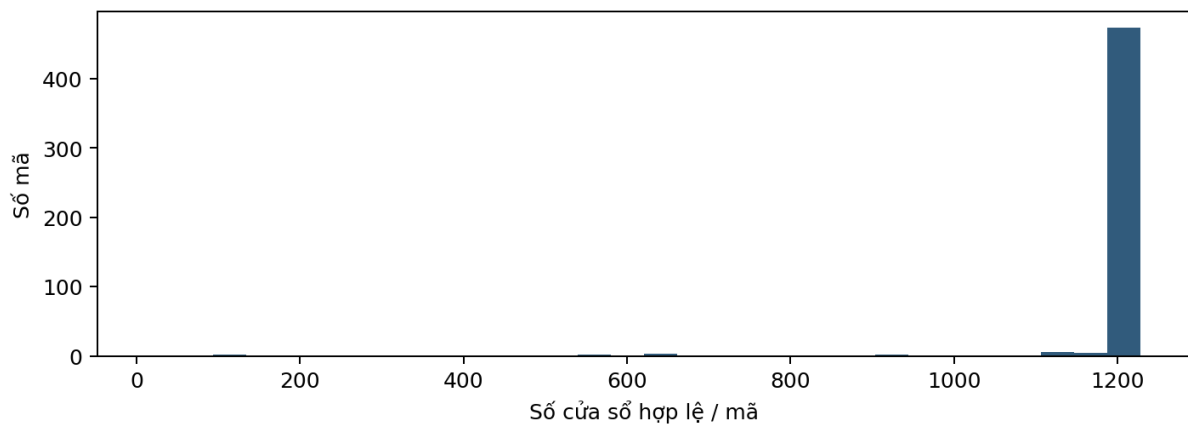
Các đại lượng trong ngày t được dùng để dự báo phiên $t+1$; high, low hoặc volume của $t+1$ không xuất hiện trong đầu vào. Cửa sổ chỉ thuộc một mã cổ phiếu. Một dòng đầu đoạn cần giá trước để tạo đặc trưng nên không được dùng như bước đủ thông tin.

Khung thời gian là ba mươi phiên quan sát liên tiếp, không phải ba mươi ngày lịch. Target là return từ giá cuối cửa sổ đến giá ở quan sát kế tiếp còn hợp lệ. Tên gọi một phiên quan sát tiếp theo giữ rõ giới hạn khi dữ liệu có ngày thiếu.

3.8. S&P 500: phân bố và độ phủ



Hình 4. Phân bố log return và ba chuỗi giá chuẩn hóa được chọn trước để minh họa.



Hình 5. Số cửa sổ hợp lệ theo từng mã cổ phiếu.

Có 11 target với độ lớn log return vượt $\log(1,5)$. Ngưỡng được dùng để thống kê, không loại nhãn đánh giá. Sai số lớn có thể liên quan biến động thị trường, corporate action hoặc chất lượng nguồn; bộ dữ liệu hiện tại không đủ xác nhận từng nguyên nhân.

Histogram return giới hạn khung hiển thị ở $\pm 0,1$ để thấy phần tập trung; đó không phải cắt ngọn dữ liệu khi huấn luyện hoặc tính metric. Mật độ được chuẩn hóa trong khung hiển thị. Đồ thị giá chuẩn hóa giúp so hình dạng nhưng không thay thế đánh giá sai số return.

3.9. Tập chia và thống kê chuẩn hóa

Dataset	Train	Validation	Test
Retailrocket	747842	122917	118192
S&P 500	414013	94249	95007

Bảng 18. Số mẫu dự đoán sau tạo cửa sổ.

Retailrocket: ranh giới validation bắt đầu 2015-08-08, test bắt đầu 2015-08-29.

S&P 500: ranh giới validation bắt đầu 2016-08-09, test bắt đầu 2017-05-10.

$$x'_j = (x_j - \mu_{j, \text{train}}) / \sigma_{j, \text{train}}$$

Ranh giới lấy theo khoảng 70% và 85% các ngày quan sát khác nhau. Tỷ lệ số dòng vì vậy không nhất thiết là 70/15/15. Nhãn được phân tập theo thời điểm mục tiêu, không theo vị trí tùy ý trong CSV.

Chúng khoán cho phép cửa sổ validation/test chứa lịch sử trước ranh giới vì lịch sử đó đã có tại thời điểm dự báo. Điều bị cấm là target tương lai tham gia học hoặc thống kê chuẩn hóa. Retailrocket bổ sung quy tắc không chia đôi một phiên.

Chuẩn hóa được fit trên các hàng đặc trưng ở giai đoạn train, trước khi lấy các cửa sổ lặp chồng lên nhau. Nhờ vậy một dòng raw không bị nhân trọng số chỉ vì xuất hiện trong nhiều cửa sổ. Biến có độ lệch chuẩn gần không dùng thang một để tránh chia cho không.

CHƯƠNG 4. KIẾN TRÚC VÀ HAI FRAMEWORK

Thành phần	Retailrocket	S&P 500
Đầu vào	$B \times 20 \times 10$	$B \times 30 \times 7$
Độ dài thật	1 đến 20	30
RNN	32 chiều, tanh	32 chiều, tanh
Đầu tuyến tính	$32 \rightarrow 3$	$32 \rightarrow 1$
Tham số học	1.475	1.313

Bảng 19. Kiến trúc chung cho hai cách triển khai.

Mô hình chỉ có một lớp RNN; không thêm cơ chế cổng hoặc mạng hai chiều. Mạng hai chiều xử lý cả hai hướng của một cửa sổ đã biết nhưng tạo thêm biến thiết kế; nghiên cứu hiện tại giữ đường truy hồi từ quá khứ đến hiện tại để giải thích và đối chiếu rõ ràng.

Hàm xây dựng được tách khỏi bộ đọc dữ liệu và bộ huấn luyện. Hai backend nhận cùng mảng float32, cùng độ dài thật và cùng nhãn. Dataset được chuẩn bị một lần, không chia lại ngẫu nhiên ở từng notebook.

Khởi tạo chung được sinh bằng NumPy: ma trận đầu vào và đầu ra dùng phân bố đều theo kích thước, ma trận hồi tiếp trực giao từ QR, bias bằng không. Seed không tự bảo đảm hai framework tạo cùng trọng số nếu mỗi framework dùng bộ sinh riêng; vì vậy bài truyền trực tiếp các ma trận chung.

4.2. RNN trong PyTorch

```
rnn = torch.nn.RNN(input_dim, 32,
                   batch_first=True, nonlinearity="tanh")
rnn.bias_hh_l0.requires_grad_(False)
packed = torch.nn.utils.rnn.pack_padded_sequence(
    x, lengths.cpu(), batch_first=True, enforce_sorted=False)
sequence_output, hidden = rnn(packed)
logits = head(hidden[-1])
```

Mã 9. Đồ thị PyTorch và đóng gói chuỗi có độ dài khác nhau.

`batch_first=True` giữ tensor theo thứ tự $B \times T \times D$. Packed sequence bỏ các bước ngoài `lengths`; `hidden[-1]` là trạng thái cuối của lớp RNN duy nhất. `enforce_sorted=False` cho phép đầu vào không sắp giảm dần độ dài và framework khôi phục thứ tự mẫu.

PyTorch lưu ma trận đầu vào theo $H \times D$ và ma trận hồi tiếp theo $H \times H$ với quy ước nhân chuyển vị. Khi gán trọng số NumPy ở dạng vector hàng, `W_x` và `W_h` phải chuyển vị. Thiếu chuyển vị có thể gây lỗi hình dạng, hoặc tạo mô hình khác nếu các chiều tình cờ bằng nhau.

Trong adapter huấn luyện, `bias_hh` được gán bằng không rồi cố định; `bias_ih` được học. Đầu tuyến tính tạo logits hoặc một giá trị hồi quy, không gán sẵn softmax để hàm loss nhận logits ổn định số.

4.3. Một bước cập nhật PyTorch

```
optimizer.zero_grad(set_to_none=True)
output = model(x, lengths)
losses = torch.nn.functional.cross_entropy(
    output, labels, reduction="none")
loss = (losses * class_weight[labels]).mean()
loss.backward()
torch.nn.utils.clip_grad_norm_(trainable_parameters, 1.0)
optimizer.step()
```

Mã 10. Bước học phân loại; đầu hồi quy thay loss bằng MSE.

zero_grad xóa gradient tích lũy từ batch trước. backward áp dụng quy tắc đạo hàm trên đồ thị forward hiện tại. Gradient clipping được thực hiện sau backward và trước optimizer.step, nên giới hạn vector gradient dùng cho bước cập nhật.

Adam duy trì moment bậc một và bậc hai cùng số bước. Chỉ lưu state_dict của mô hình đủ để suy luận nhưng không đủ để tiếp tục Adam đúng trạng thái. Checkpoint của nghiên cứu lưu cả model, optimizer và trạng thái bộ sinh ngẫu nhiên PyTorch.

Loss epoch là tổng loss batch nhân số mẫu rồi chia tổng mẫu. Batch cuối nhỏ không được nhận trọng số ngang một batch đầy đủ. Bài không dùng gradient accumulation hoặc mixed precision để giữ quy trình số học dễ đối chiếu.

4.4. SimpleRNN trong Keras

```
sequence = tf.keras.Input((length, input_dim))
sizes = tf.keras.Input((), dtype="int32")
mask = tf.sequence_mask(sizes, maxlen=length)
hidden = tf.keras.layers.SimpleRNN(
    32, activation="tanh")(sequence, mask=mask)
output = tf.keras.layers.Dense(output_dim)(hidden)
model = tf.keras.Model([sequence, sizes], output)
```

Mã 11. Keras Functional API với lengths làm đầu vào tường minh.

SimpleRNN nhận hai ma trận kernel và recurrent_kernel cùng một vector bias. Với return_sequences=False mặc định, lớp trả trạng thái cuối hợp lệ sau khi xét mask. Mặt nạ được xây dựng theo số bước thật, không dựa trên giá trị của đặc trưng.

Functional API biểu diễn rõ hai đầu vào: dữ liệu và lengths. Trong phiên bản TensorFlow/Keras dùng cho thực nghiệm, tf.sequence_mask tạo thao tác đồ thị tương thích với tensor Keras. Mã và phiên bản được lưu cùng hồ sơ môi trường.

Bản triển khai dùng tf.keras đi kèm TensorFlow 2.10.1 trong môi trường Anaconda đã có trên máy. Nó không ngầm chuyển sang Keras 3 hay backend khác trong các lượt so sánh.

4.5. GradientTape và checkpoint Keras

```
with tf.GradientTape() as tape:
    logits = model([x, lengths], training=True)
    losses = tf.nn.softmax_cross_entropy_with_logits(
        labels=tf.one_hot(labels, 3), logits=logits)
    loss = tf.reduce_mean(losses * tf.gather(weights, labels))
gradients = tape.gradient(loss, model.trainable_variables)
gradients, norm = tf.clip_by_global_norm(gradients, 1.0)
optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

Mã 12. Bước học Keras theo cùng mẫu số và trọng số lớp.

GradientTape ghi các phép biến đổi cần thiết để lấy đạo hàm theo biến học. `tf.function` biên dịch bước học để giảm chi phí gọi Python; lần gọi đầu có chi phí tạo đồ thị nên thời gian chạy thử nhỏ không thể ngoại suy đơn giản sang toàn bộ tập.

`tf.train.Checkpoint` lưu mô hình và optimizer, bao gồm các biến moment đã được tạo. Con trỏ `state.json` chỉ được cập nhật sau khi checkpoint hoàn tất. Nếu ngắt khi đang ghi, lần chạy sau vẫn đọc epoch hoàn chỉnh trước đó.

Bài không dùng dropout trong mô hình thực nghiệm. Thứ tự batch được sinh lại từ seed cộng số epoch. Thiết kế này giảm trạng thái ngẫu nhiên phải theo dõi khi khởi động lại; kết quả tái lập vẫn phụ thuộc điều kiện backend và thiết bị.

4.6. Đối chiếu cài đặt và giới hạn tương đương

Framework	Sai số forward lớn nhất	Sai số bước tiếp tục
pytorch	1.88e-07	0.00e+00
keras	2.86e-07	0.00e+00

Bảng 20. Kiểm tra CPU trên ví dụ nhỏ với cùng trọng số.

Hai cách triển khai có cùng hàm toán học khi trọng số được ánh xạ đúng. Sai số nhỏ còn lại xuất phát từ số thực hữu hạn và thứ tự phép tính. Tương đương forward không đồng nghĩa mọi cập nhật Adam và mọi kernel GPU khớp bitwise.

Cùng epsilon và learning rate chưa loại bỏ mọi khác biệt triển khai optimizer. Vì vậy báo cáo đối chiếu chất lượng, đường học và chi phí của hai framework trên cùng thiết kế, không coi chênh lệch nhỏ là bằng chứng một framework có khả năng biểu diễn tốt hơn.

Giữ chung	Có thể khác
Dữ liệu và ranh giới thời gian	Kernel và thứ tự cộng số thực
Trọng số ban đầu từ NumPy	Chi tiết thực thi optimizer
Loss, trọng số lớp, clipping	Chi phí packed sequence/masking
Cửa sổ, hidden size, seed	Thời gian khởi tạo đồ thị

Bảng 21. Điều kiện so sánh và các nguồn khác biệt còn lại.

CHƯƠNG 5. THIẾT KẾ THỰC NGHIỆM

Thành phần	Thiết lập
Dataset	Retailrocket; S&P 500
Framework	PyTorch; Keras/TensorFlow
Mô hình	Simple RNN một lớp, tanh, H=32
Số lượt chính	$2 \times 2 = 4$
Seed	42
Batch size	256
Epoch tối đa	20
Learning rate	0.001
Clipping norm	1
Early stopping	Patience 4; min_delta 0,00001

Bảng 22. Các điều kiện chính của thực nghiệm.

Một seed được dùng cho bốn cấu hình chính để tập trung vào RNN và kiểm soát thời gian chạy. Không báo cáo trung bình hay độ lệch chuẩn giữa các seed khi chỉ có một lượt cho mỗi tổ hợp. Kết quả phản ánh cấu hình cụ thể, chưa định lượng đầy đủ biến thiên khởi tạo.

Mỗi lượt chạy trong tiến trình riêng để tránh giữ đồng thời hai framework trên GPU. Notebook minh họa chạy CPU và mặc định không khởi động huấn luyện đầy đủ. Hai loại hoạt động tạo đầu ra khác nhau và được lưu ở thư mục riêng.

5.2. Validation, chọn checkpoint và test

Nhiệm vụ	Tiêu chí chọn	Metric bổ sung
Retailrocket	Macro-F1 validation lớn hơn	Cross-entropy, accuracy, từng lớp, AP
S&P 500	RMSE return validation nhỏ hơn	MAE, hướng thay đổi, sai số giá

Bảng 23. Quy tắc chọn mô hình trước khi đánh giá test.

Một checkpoint mới được chọn khi tiêu chí cải thiện quá $\min_delta$ so với checkpoint đang giữ. Nếu không cải thiện qua bốn epoch liên tiếp, lượt dừng sớm. Sau khi kết thúc ngân sách hoặc dừng sớm, checkpoint được chọn được tải lại rồi mới sinh dự đoán test.

Loss train của Retailrocket có trọng số lớp, còn cross-entropy validation không có trọng số. Hai đường vì vậy phản ánh các mục tiêu khác nhau; khoảng cách giữa chúng không chỉ đo quá khớp. Với chứng khoán, loss train dùng target đã chia thang, còn RMSE validation giữ đơn vị log return gốc.

Test không dùng để chọn epoch, trọng số lớp, scaler hoặc siêu tham số. Những phân tích sau test mang tính mô tả; nếu được dùng để cải tiến mô hình, vòng đánh giá sau cần một thiết kế kiểm tra độc lập phù hợp.

5.3. Phương pháp đối chứng

Dataset	Đối chứng	Thông tin sử dụng
Retailrocket	Prior toàn train	Tần suất ba hành vi
Retailrocket	Markov bậc một	Hành vi gần nhất và bảng chuyển train
S&P 500	Zero-return	Giá dự báo bằng giá gần nhất
S&P 500	Mean-return	Return trung bình toàn train

Bảng 24. Các mốc so sánh đơn giản.

Prior và Markov dùng làm tron cộng một trước khi chuyển thành xác suất, tránh log của không. Markov kiểm tra liệu trạng thái ẩn RNN có mang lại lợi ích so với việc chỉ nhớ hành vi ngay trước đó. Các xác suất đều được ước lượng từ train.

Zero-return là đối chứng quan trọng vì giá thường thay đổi tương đối ít giữa hai phiên liên tiếp. Mean-return thêm một mức trôi ước lượng từ train. Cả hai không được cập nhật bằng nhãn test trong quá trình đánh giá.

Đối chứng không được gọi là yếu chỉ vì ít tham số. Nếu phương pháp đơn giản đạt sai số thấp hơn RNN, đó là kết quả có ý nghĩa về giá trị của cấu hình hồi tiếp trong dữ liệu và ngân sách hiện tại.

5.4. Dừng, lưu và tiếp tục thực nghiệm

```
adapter.save(checkpoint_for(epoch))
state["epoch"] = epoch
state["history"].append(epoch_record)
write_json_atomic("state.json", state)
```

Mã 13. Thứ tự công bố epoch đã hoàn tất.

Checkpoint đầy đủ được lưu ở cuối epoch, sau đánh giá validation. Tập state.json được thay thế nguyên tử để chỉ trở tới trạng thái đã ghi xong. Các checkpoint gần nhất và tốt nhất được giữ; bản không còn được tham chiếu được dọn nhằm hạn chế dung lượng.

Yêu cầu tạm dừng có hiệu lực ở ranh giới epoch. Nếu tiến trình bị ngắt ngay giữa epoch, phần cập nhật chưa được công bố sẽ được thực hiện lại từ đầu epoch khi chạy tiếp. Việc này khác với giữ nguyên tiến trình trong RAM khi hệ điều hành Sleep.

Framework	Khởi động lại tiến trình	Sai khác đầu ra lớn nhất
pytorch	Đã kiểm tra	0.0
keras	Đã kiểm tra	0.0

Bảng 25. So sánh smoke hai epoch liên với dừng sau epoch thứ nhất.

Kiểm tra đạt trong môi trường hiện tại không bảo đảm kết quả giống từng bit khi đổi thiết bị, thư viện hoặc cấu hình. Các tệp config và phiên bản đi cùng checkpoint là điều kiện cần để diễn giải phép chạy tiếp.

5.5. Hồ sơ kiểm chứng chương trình

Kiểm tra	Kết quả
Tập chia theo thời gian	Hai dataset đạt
BPTT NumPy	Gradient số và giải tích khớp
Forward hai framework	Khớp ví dụ NumPy
Masking	Đổi padding không đổi dự đoán
Tiếp tục optimizer	Đã đổi chiều bước tiếp theo
Chạy lại qua tiến trình	Hai framework đạt
Notebook thực thi	10
Cell code	73

Bảng 26. Các kiểm tra chương trình đã thực hiện.

Smoke dùng tối đa 512 mẫu train, 128 mẫu validation và hai epoch. Nó kiểm tra đường đi của chương trình trên dữ liệu thật nhưng không đo chất lượng mô hình chính. Test chính thức không được dùng làm tập đánh giá của smoke.

Sau thực nghiệm chính, dự đoán được lưu cùng ID, nhãn và thời điểm để tính lại metric. Trọng số được tải lại độc lập để đổi chiều một batch. Lịch sử, cấu hình và checkpoint giúp truy vết cách chọn mô hình thay vì chỉ lưu một bảng điểm cuối.

Nguồn code, notebook và hướng dẫn môi trường được công bố tại https://github.com/namnguyen05-vn/PTHTTM_Assignment-06. Dữ liệu raw, cache và tệp tạm đặt trên ổ E, tách khỏi bộ bài nộp và lịch sử Git.

CHƯƠNG 6. KẾT QUẢ VÀ THẢO LUẬN

Các số đo trong chương này được tính trên toàn bộ test, sau khi checkpoint được chọn bằng validation. Retailrocket có 118.192 mẫu; S&P 500 có 95.007 mẫu. Mỗi cấu hình dùng seed 42. Các lượt chạy thử nhỏ chỉ phục vụ kiểm tra chương trình và không tham gia bảng kết quả.

6.1. Kết quả tổng thể trên Retailrocket

Mô hình	Accuracy (%)	Macro-F1	Log loss
pytorch	90,09	0,582107	0,340692
keras	90,11	0,581783	0,340711
Tần suất train	91,56	0,318638	0,335413
Markov bậc một	91,56	0,318638	0,288966

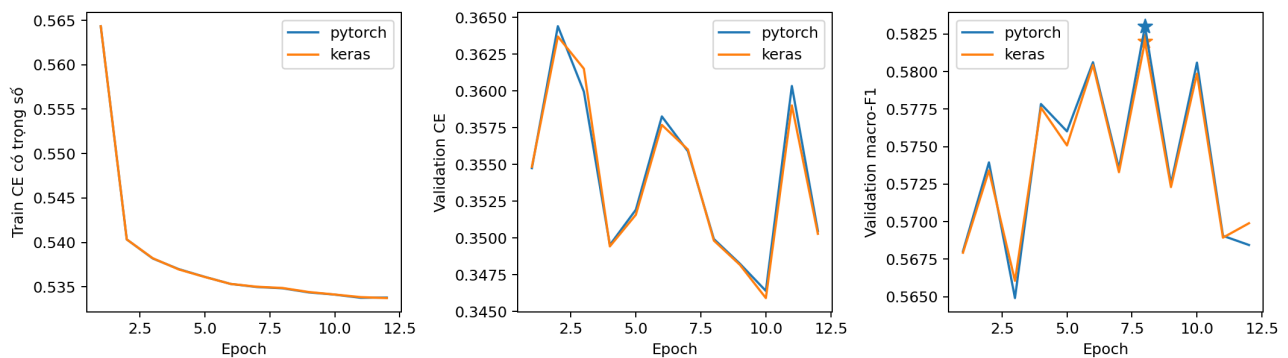
Bảng 27. Kết quả Retailrocket. Accuracy và F1 càng cao càng tốt; log loss càng thấp càng tốt.

RNN PyTorch đạt macro-F1 0,5821, cao hơn Markov 0,2635 điểm trên thang 0-1. Keras cho kết quả gần tương đương. Tuy nhiên, accuracy của cả hai RNN chỉ khoảng 90,1%, thấp hơn mức 91,56% của đối chứng luôn chọn view.

Hai đối chứng đều chọn view khi lấy xác suất lớn nhất: trong cả ba hàng của ma trận chuyển Markov, view vẫn là lớp có xác suất cao nhất. Vì vậy, accuracy và macro-F1 của chúng trùng nhau, nhưng log loss khác nhau do phân bố xác suất dự đoán khác nhau.

Markov có log loss tốt hơn RNN. Kết quả không chứng minh RNN vượt đối chứng trên mọi tiêu chí. Huấn luyện RNN dùng trọng số lớp và lựa chọn theo macro-F1; hai đối chứng dùng tần suất không gán trọng số. Sự cải thiện lớp hiếm có thể đến từ cả mục tiêu học và đặc trưng, không thể quy riêng cho bộ nhớ hồi tiếp khi chưa có thực nghiệm loại bỏ thành phần.

6.2. Đường học và lựa chọn mô hình hành vi



Hình 6. Loss train, log loss validation và macro-F1 validation; dấu sao chỉ epoch được chọn.

Framework	Epoch chọn	Macro-F1 val	Epoch dừng
pytorch	8	0,583026	12
keras	8	0,582000	12

Bảng 28. Cả hai lượt chọn epoch 8 và dừng sau epoch 12.

Loss train giảm chủ yếu trong các epoch đầu rồi thay đổi chậm. Macro-F1 validation dao động thay vì tăng đều. Bốn epoch sau epoch 8 không vượt điểm tốt nhất đủ min_delta, nên early stopping dừng ở epoch 12 và khôi phục checkpoint epoch 8.

Log loss validation thấp nhất không nằm ở epoch được chọn. Điều này phù hợp thiết kế: tiêu chí lựa chọn là macro-F1, không phải cross-entropy. Dừng test để đổi sang một epoch có kết quả đẹp hơn sẽ phá vỡ vai trò đánh giá độc lập của test.

Loss train là cross-entropy có trọng số lớp; log loss validation không có trọng số. Do khác định nghĩa, không diễn giải trực tiếp khoảng cách giữa hai đường như độ chênh train-test của cùng một loss.

6.3. Precision, recall và F1 của từng hành vi

Framework	Lớp	Precision	Recall	F1	Số mẫu
pytorch	view	0,9478	0,9499	0,9489	108211
pytorch	addtocart	0,3929	0,2735	0,3225	7683
pytorch	transaction	0,3616	0,6919	0,4750	2298
keras	view	0,9478	0,9501	0,9490	108211
keras	addtocart	0,3943	0,2716	0,3217	7683
keras	transaction	0,3607	0,6941	0,4747	2298

Bảng 29. Đánh giá riêng ba loại hành vi trên cùng test.

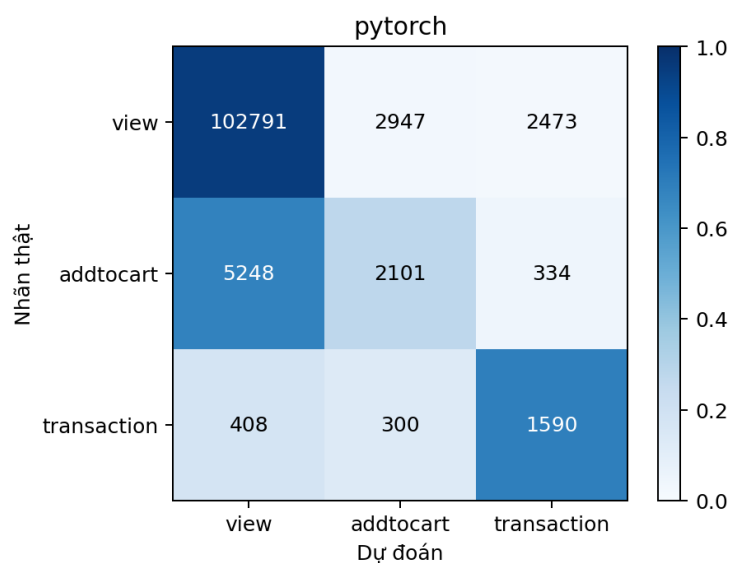
Precision là tỷ lệ dự đoán đúng trong các mẫu được gán vào một lớp; recall là tỷ lệ tìm đúng trong toàn bộ mẫu thật của lớp đó. F1 là trung bình điều hòa của precision và recall. Macro-F1 lấy trung bình F1 của ba lớp với trọng số bằng nhau, nên lớp view không thể che lấp toàn bộ lỗi ở lớp hiếm.

Với PyTorch, transaction có recall khoảng 69,19% nhưng precision chỉ 36,16%. Mô hình tìm được nhiều giao dịch hơn đối chứng luôn-view, đồng thời tạo nhiều cảnh báo giao dịch sai. Addtocart có recall khoảng 27,35%, nghĩa là phần lớn hành vi thêm giỏ vẫn chưa được nhận diện đúng.

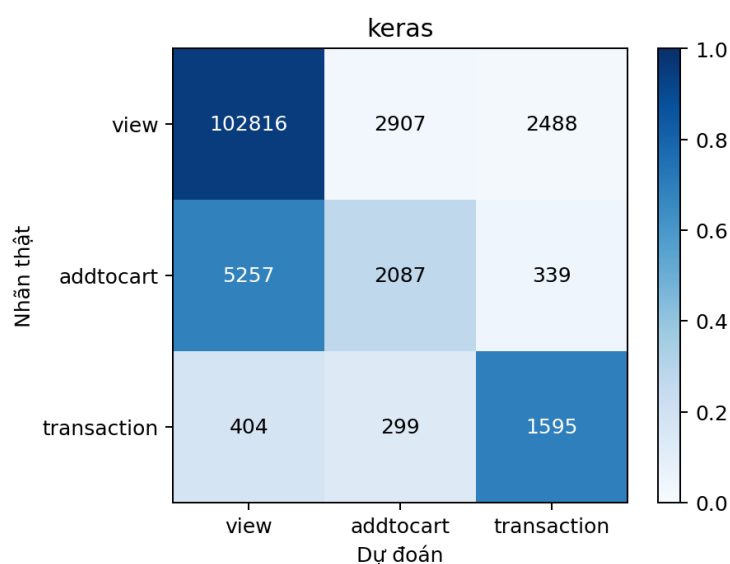
F1 của view gần 0,949, cao hơn đáng kể hai lớp còn lại. Kết quả phù hợp tính mất cân bằng của dữ liệu và cho thấy macro-F1 khoảng 0,582 không đồng nghĩa mọi lớp đều có F1 ở mức đó.

Các chỉ số đo loại sự kiện kế tiếp trong các mẫu có nhãn. Chúng không đo số đơn hàng, doanh thu hoặc xác suất mua của những khách đã rời phiên.

6.4. Ma trận nhầm lẫn và dạng lỗi



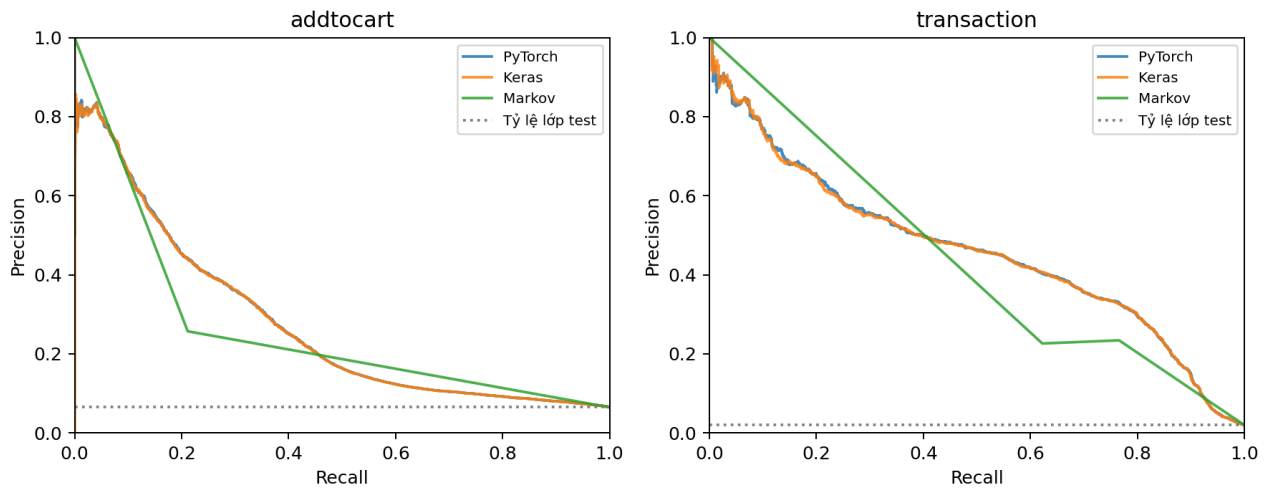
Hình 7. Ma trận pytorch: màu chuẩn hóa theo nhãn thật, số trong ô là số mẫu.



Hình 8. Ma trận keras: màu chuẩn hóa theo nhãn thật, số trong ô là số mẫu.

Ở PyTorch, 5.248/7.683 mẫu addtocart bị gán thành view. Trong 4.397 dự đoán transaction, chỉ 1.590 mẫu đúng; 2.473 mẫu thực chất là view và 334 là addtocart. Hai ma trận gần giống nhau, cho thấy hai cách triển khai mắc các dạng lỗi tương tự.

6.5. Khả năng xếp hạng các lớp hiếm



Hình 9. Đường precision-recall trên test; đường ngang là tỷ lệ xuất hiện lớp.

Mô hình	AP addtocart	AP transaction
PyTorch	0,275497	0,460469
Keras	0,274927	0,459440
Markov	0,105927	0,178815
Tần suất train	0,065004	0,019443

Bảng 30. Average precision (AP) cho bài toán mỗi lớp so với phần còn lại.

Average precision tổng hợp precision theo các mức tăng recall khi thay đổi ngưỡng điểm số. AP xem xét khả năng xếp hạng mẫu của từng lớp, khác với F1 tại quyết định argmax. Trong bài này, AP được tính bằng `average_precision_score`; không đồng nhất với mọi cách lấy diện tích hình thang dưới đường PR.

RNN đạt AP cao hơn Markov ở cả addtocart và transaction. Đây là bằng chứng bổ sung về khả năng ưu tiên các mẫu thuộc lớp hiếm, dù log loss tổng thể chưa tốt hơn. Hai nhận xét không mâu thuẫn vì chúng đo các khía cạnh khác nhau của dự đoán.

Đường PR được dùng để mô tả kết quả sau thực nghiệm. Không chọn ngưỡng triển khai bằng test và không báo cáo một ngưỡng tối ưu tìm trên test như kết quả đánh giá độc lập.

6.6. Kết quả dự báo log return S&P 500

Mô hình	MAE return	RMSE return	Đúng hướng (%)
PyTorch	0,01001167	0,01553288	50,17
Keras	0,01001166	0,01553286	50,17
Zero return	0,00975611	0,01529663	0,83
Mean return train	0,00973583	0,01529150	53,09

Bảng 31. Sai số trên 95.007 mẫu test; return dùng thang logarit gốc.

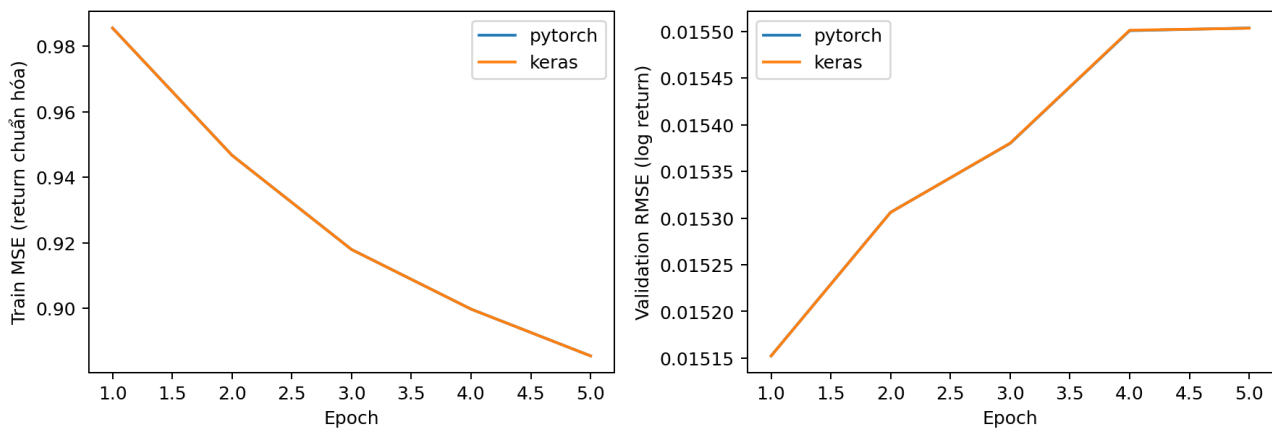
RNN PyTorch có RMSE 0,01553288, cao hơn đối chứng trung bình train khoảng 1,58%. Keras gần như trùng kết quả. MAE cũng cao hơn cả hai đối chứng. Vì vậy, cấu hình RNN này chưa tạo giá trị dự báo bổ sung theo các thước đo sai số đã chọn.

Độ đúng hướng của RNN khoảng 50,17%, thấp hơn 53,09% của dự đoán luôn có return dương bằng trung bình train. Test có 53,09% mẫu tăng, 46,08% giảm và 0,83% không đổi. Do mất cân bằng nhẹ về hướng, mức 50% không phải đối chứng duy nhất.

Directional accuracy trong mã dùng $\text{sign}(\text{dự đoán}) = \text{sign}(\text{thực})$. Zero return luôn dự đoán dấu 0, nên chỉ đúng ở mẫu không đổi và đạt khoảng 0,83%. Số này không có nghĩa đối chứng zero-return kém về RMSE; hai chỉ số phục vụ hai câu hỏi khác nhau.

Mỗi mẫu có trọng số như nhau trong chỉ số gộp. Các mẫu cùng mã và cùng ngày thị trường có thể phụ thuộc lẫn nhau; không coi 95.007 mẫu là các phép thử độc lập để suy ra ý nghĩa thống kê.

6.7. Diễn biến huấn luyện dự báo cổ phiếu



Hình 10. Loss train trên return đã chuẩn hóa và RMSE validation trên return gốc.

Framework	Epoch chọn	RMSE val tại epoch chọn	Epoch dừng
pytorch	1	0,01515236	5
keras	1	0,01515232	5

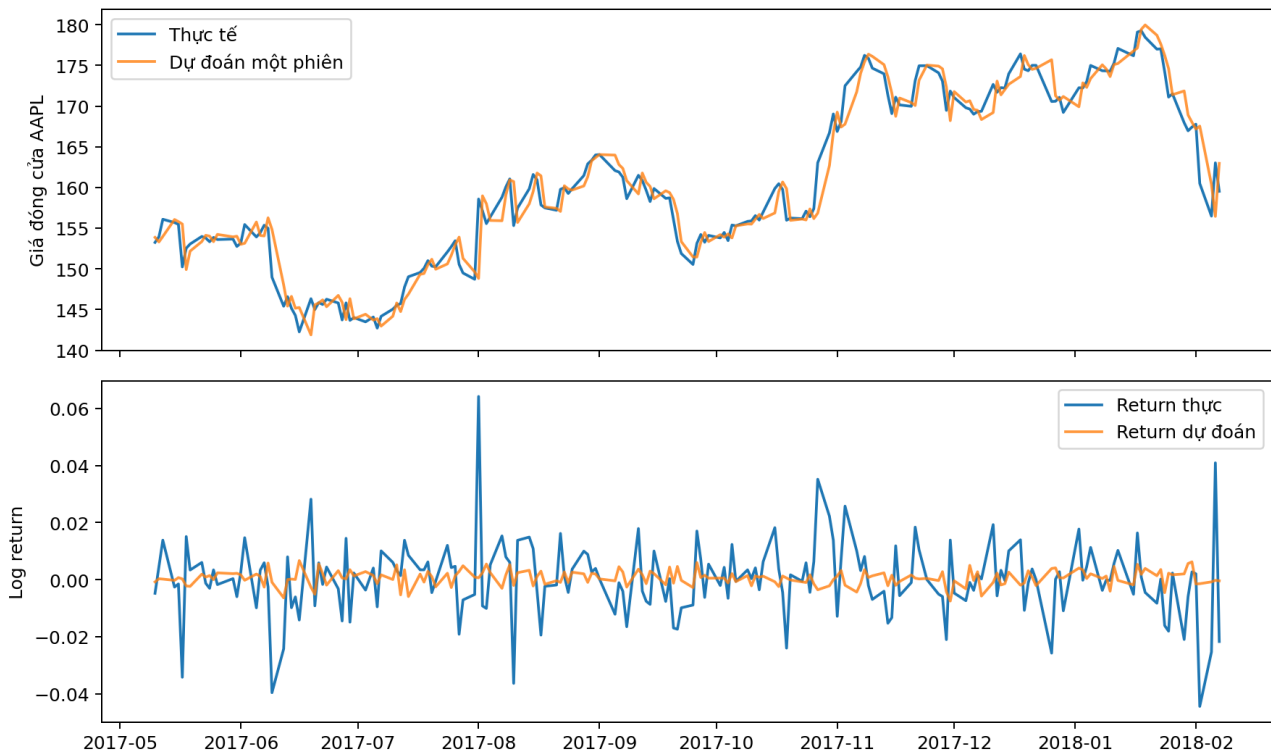
Bảng 32. Hai lượt đều chọn epoch 1 và dừng sau epoch 5.

Loss train tiếp tục giảm từ khoảng 0,986 xuống 0,886, trong khi RMSE validation tăng sau epoch 1. Mẫu hình này phù hợp với việc cải thiện độ khớp train không chuyển thành cải thiện trên giai đoạn kế tiếp. Nó có thể liên quan tới quá khớp và khác biệt phân phối; riêng các đường học chưa phân biệt được hai nguyên nhân.

Bộ huấn luyện đã khôi phục checkpoint epoch 1 trước khi dự đoán test. Kết quả không phải dự đoán của epoch 5. Early stopping được kiểm tra lại trực tiếp từ history, min_delta và patience, đồng thời checkpoint được nạp trong tiến trình mới.

Không kéo dài huấn luyện hoặc chỉnh cấu hình theo test sau khi quan sát kết quả này. Một nghiên cứu tiếp theo có thể kiểm tra cửa sổ hoặc số chiều ẩn của Simple RNN bằng nhiều giai đoạn validation, sau đó cần một khoảng test chưa được sử dụng.

6.8. Giá và return dự báo: pytorch



Hình 11. AAPL trên test: giá đóng cửa và log return của phiên quan sát kế tiếp.

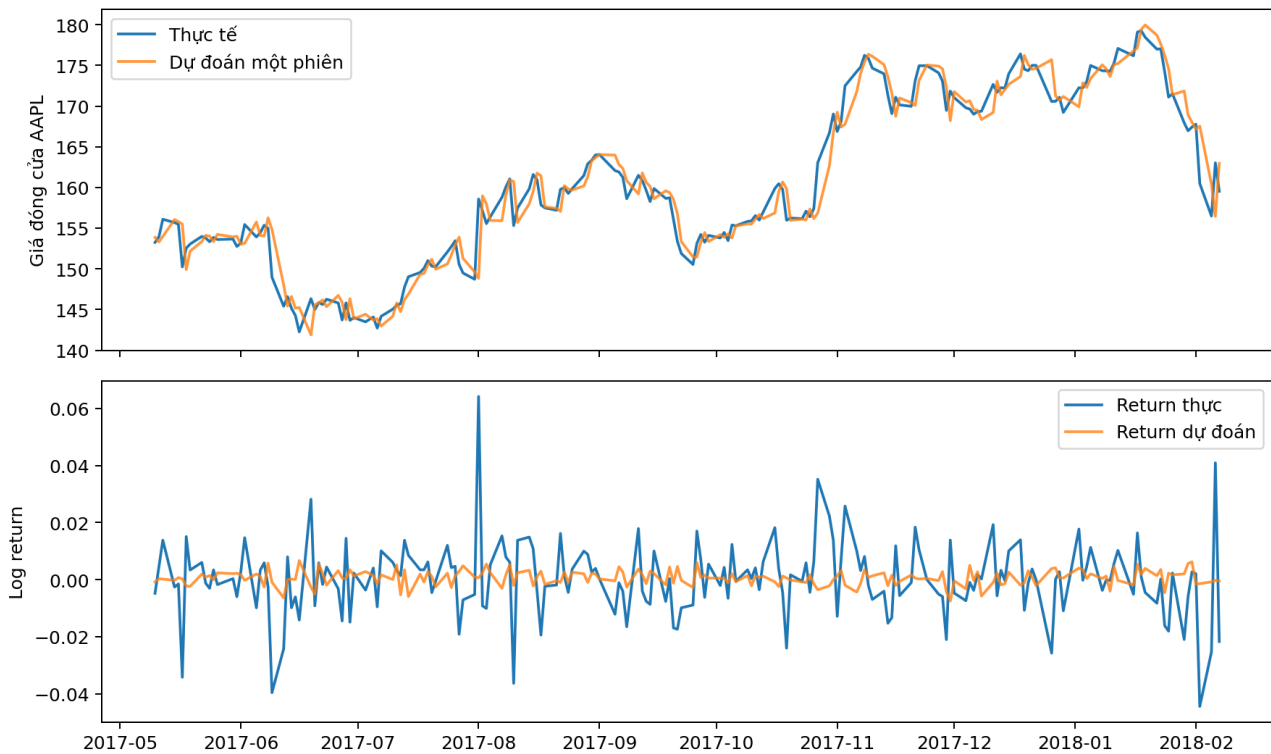
AAPL được quy định là mã minh họa trong mã vẽ trước khi chạy thực nghiệm. Hình chỉ mô tả một mã, còn các metric chính được tính trên toàn bộ tập test nhiều mã.

Giá dự báo được tái dựng từ giá thật cuối cửa sổ nhân với $\exp(\text{return dự báo})$. Vì giá phiên liền trước đã gần giá phiên kế tiếp, đường giá có thể bám sát thực tế ngay cả khi dự báo return ít thông tin. Panel return giúp nhận ra giới hạn này.

Trên toàn bộ test, MAE giá của pytorch là 0,981075 và RMSE giá là 2,413370, đều cao hơn đối chứng zero-return lần lượt 0,957937 và 2,388390. Sai số giá gộp chịu ảnh hưởng của mức giá khác nhau giữa các mã.

Đây là dự đoán một bước có cập nhật quan sát thật ở mỗi cửa sổ. Không diễn giải hình thành dự báo nhiều bước liên tiếp hoặc bằng chứng về lợi nhuận giao dịch.

6.9. Giá và return dự báo: keras



Hình 12. AAPL trên test: giá đóng cửa và log return của phiên quan sát kế tiếp.

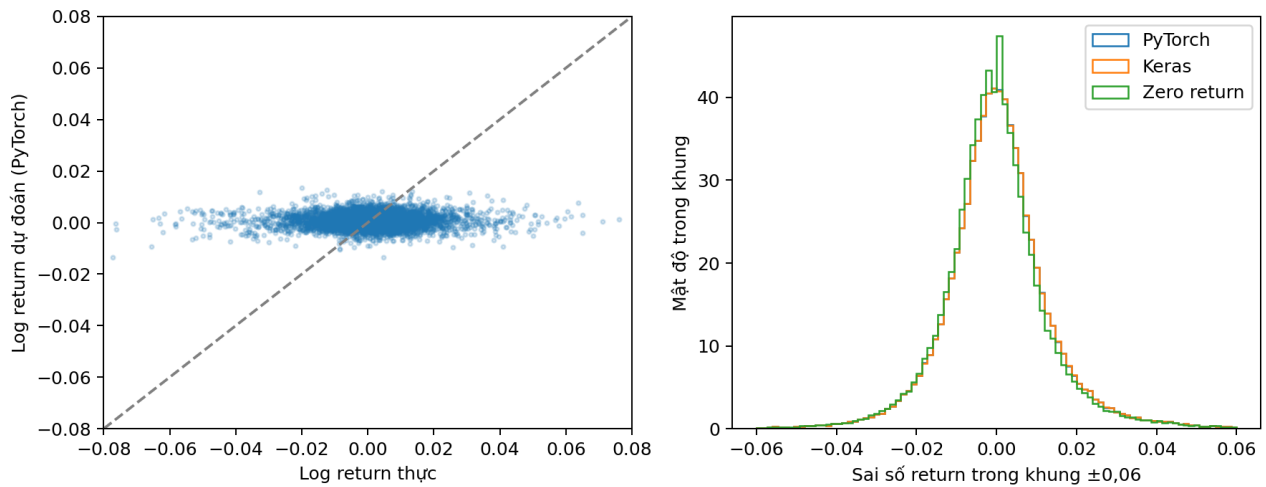
AAPL được quy định là mã minh họa trong mã vẽ trước khi chạy thực nghiệm. Hình chỉ mô tả một mã, còn các metric chính được tính trên toàn bộ tập test nhiều mã.

Giá dự báo được tái dựng từ giá thật cuối cửa sổ nhân với $\exp(\text{return dự báo})$. Vì giá phiên liền trước đã gần giá phiên kế tiếp, đường giá có thể bám sát thực tế ngay cả khi dự báo return ít thông tin. Panel return giúp nhận ra giới hạn này.

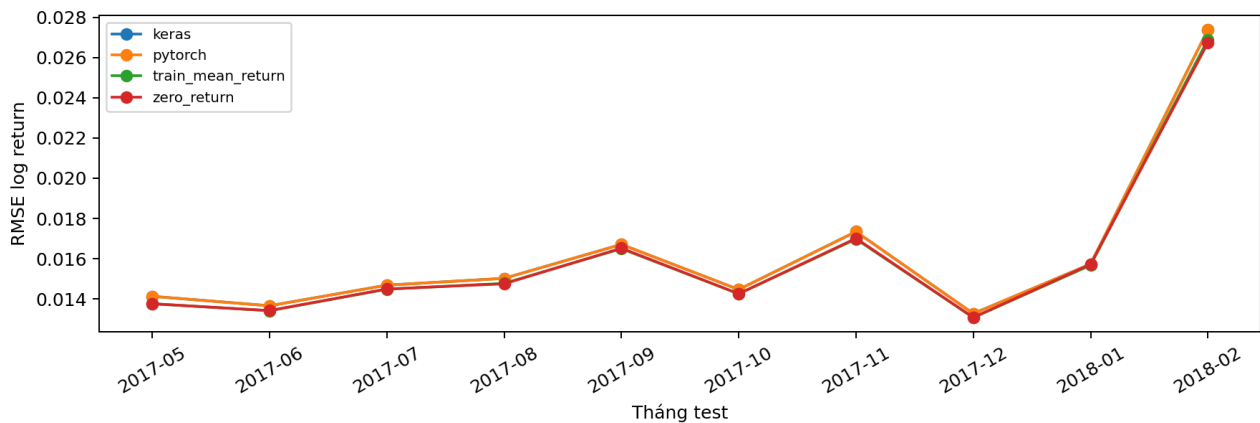
Trên toàn bộ test, MAE giá của keras là 0,981076 và RMSE giá là 2,413375, đều cao hơn đối chứng zero-return lần lượt 0,957937 và 2,388390. Sai số giá gộp chịu ảnh hưởng của mức giá khác nhau giữa các mã.

Đây là dự đoán một bước có cập nhật quan sát thật ở mỗi cửa sổ. Không diễn giải hình thành dự báo nhiều bước liên tiếp hoặc bằng chứng về lợi nhuận giao dịch.

6.10. Sai số theo phân phối và thời gian



Hình 13. Trái: 6.000 mẫu chọn với seed 42, khung return $\pm 0,08$. Phải: histogram sai số trong khung $\pm 0,06$.



Hình 14. RMSE theo tháng test. Tháng đầu và cuối chỉ bao gồm phần thời gian có trong test.

Độ lệch chuẩn return dự đoán PyTorch là 0,002689, nhỏ hơn độ lệch chuẩn return thật 0,015291. Mô hình tạo tín hiệu dao động hẹp, không tái hiện tốt các biến động lớn. Các khung hình chỉ giới hạn vùng hiển thị; metric vẫn dùng toàn bộ mẫu.

Ở cả mười tháng được quan sát, RMSE của hai RNN đều cao hơn zero-return. Tháng 02/2018 có sai số lớn nhất, nhưng chỉ có 2.525 mẫu và không bao trùm cả tháng. Phân rã này mô tả độ ổn định theo thời gian, không thay thế đánh giá ở một giai đoạn độc lập.

6.11. Hai framework và chi phí thực nghiệm

Dataset	Framework	Epoch chạy	Epoch chọn	Tổng giây	Giây/epoch
retailrocket	pytorch	12	8	500,0	41,7
retailrocket	keras	12	8	1007,7	84,0
sp500	pytorch	5	1	131,8	26,4
sp500	keras	5	1	304,9	61,0

Bảng 33. Thời gian epoch hoàn tất gồm huấn luyện và tính validation.

Tổng thời gian được ghi trong history của bốn lượt là 32,41 phút. Trong lần chạy này, Keras mất khoảng 2,02 lần thời gian PyTorch trên Retailrocket và 2,31 lần trên S&P 500. Đây là phép đo tại máy và phiên bản cụ thể, không phải xếp hạng tốc độ phổ quát.

Runtime ghi nhận CUDA cho cả bốn lượt, PyTorch 2.5.1+cu121 và TensorFlow 2.10.1. PyTorch dùng chuỗi đóng gói theo độ dài; Keras dùng mặt nạ độ dài. Cách tổ chức phép tính khác nhau dù kiến trúc và trọng số khởi tạo tương ứng.

Đối chiếu trên cùng ID test	Giá trị
Tỷ lệ nhãn hành vi trùng nhau (%)	99,83
Sai khác xác suất trung bình	0,00163846
Sai khác xác suất lớn nhất	0,23988628
Sai khác return trung bình	0,0000025924
Sai khác return lớn nhất	0,0000159966

Bảng 34. So sánh dự đoán sau huấn luyện của PyTorch và Keras.

Sai khác trung bình nhỏ không bảo đảm mọi mẫu đều giống nhau: chênh lệch xác suất lớn nhất ở Retailrocket gần 0,24. Macro-F1 hai lượt chỉ lệch khoảng 0,00032, còn RMSE cổ phiếu lệch khoảng $2,28 \times 10^{-8}$. Với một seed, không có cơ sở tuyên bố ưu thế thống kê của một framework.

6.12. Kiểm chứng đầu ra và khả năng truy vết

Dataset	Framework	Mẫu tải lại	Sai khác lớn nhất
retailrocket	pytorch	256	0,00000000
retailrocket	keras	256	0,00000000
sp500	pytorch	256	0,00000000
sp500	keras	256	0,00000000

Bảng 35. Nạp checkpoint tốt nhất trong tiến trình mới và đối chiếu dự đoán đã lưu.

Cả 4 lượt vượt qua kiểm tra tính lại metric từ outputs.npz và nhãn gốc. ID, thời điểm, nhãn và dự đoán trong CSV được đối chiếu với mảng dữ liệu; hash metadata được kiểm tra với cấu hình đã lưu. Epoch chọn, epoch dừng và tổng thời gian được tái dựng từ history.

Kiểm tra checkpoint dùng 256 mẫu đầu test của mỗi lượt, không khẳng định đã chạy lại suy luận toàn bộ test. Sai khác lớn nhất bằng 0 trên các batch kiểm tra trong cùng họ thiết bị. Điều này bổ sung bằng chứng về tính nhất quán của tệp đã lưu, không thay thế kiểm tra chất lượng dự báo.

Nhóm tệp	Vai trò
metrics.json; history.csv	Chỉ số test và diễn biến từng epoch
outputs.npz; predictions.csv	Dự đoán kèm ID, nhãn và thời điểm
config.json; state.json; checkpoints	Cấu hình, lựa chọn epoch và trọng số/optimizer
experiment_summary.csv; baselines.json	Bảng tổng hợp và các đối chứng
provenance_verification.json	Kiểm tra lịch sử, nguồn dữ liệu và CSV

Bảng 36. Các bằng chứng đi cùng notebook và báo cáo.

Notebook lưu đầu ra thực thi, chia cell ngắn theo từng bước. README trình bày môi trường và cách tái lập, còn báo cáo tập trung vào phương pháp, số liệu, diễn giải và giới hạn. Dữ liệu gốc được dẫn bằng liên kết Kaggle và mã tải, tách khỏi repository mã nguồn.

CHƯƠNG 7. KẾT LUẬN

Nghiên cứu đã xây dựng và kiểm chứng RNN cơ bản dưới góc nhìn hàm: trạng thái ẩn là kết quả lặp của phép biến đổi affine và tanh; các bước dùng chung trọng số; gradient được truyền qua đồ thị trải theo thời gian. Ví dụ NumPy, kiểm tra sai phân hữu hạn và hai triển khai thư viện liên kết công thức với mã thực thi.

Hai tập dữ liệu lớn được chuyển thành bài toán chuỗi có mối quan sát rõ ràng. Retailrocket dùng lịch sử hành vi trong phiên để dự đoán loại sự kiện kế tiếp; S&P 500 dùng cửa sổ 30 phiên để dự đoán log return. Chia tập theo thời gian, chuẩn hóa chỉ dựa trên train và tách dữ liệu giữa các thực thể tạo cơ sở cho đánh giá có thể truy vết.

Bốn lượt huấn luyện hoàn tất và được kiểm tra từ dự đoán, lịch sử và checkpoint. Hai framework cho chất lượng rất gần nhau, đồng thời khác nhau về thời gian chạy trong môi trường được khảo sát. Kết quả một seed chưa đủ để khẳng định ưu thế chung của PyTorch hoặc Keras.

Trên hành vi khách hàng, RNN có trọng số lớp cải thiện macro-F1 và khả năng xếp hạng hai hành vi hiếm, đổi lại accuracy thấp hơn đối chứng luân-view và log loss kém hơn Markov. Chưa có thực nghiệm loại bỏ thành phần để tách tác dụng của lịch sử, đặc trưng và trọng số lớp. Vì vậy, kết luận giới hạn ở cấu hình và thước đo đã đo.

Trên cổ phiếu, mô hình chưa vượt dự đoán return bằng không hoặc trung bình train về sai số. Đường giá một bước gần thực tế chủ yếu phải được đọc cùng giá phiên trước đã biết và sai số return. Kết quả âm này là một phát hiện của thực nghiệm, không phải lý do thay thế test hoặc chỉ chọn hình minh họa thuận lợi.

Hướng mở rộng trong phạm vi Simple RNN gồm đánh giá cuốn chiếu nhiều giai đoạn, lặp nhiều seed, khảo sát độ dài cửa sổ và số chiều ẩn bằng validation, cùng đối chứng được gán trọng số lớp tương ứng. Những nội dung này chưa được thực hiện trong bộ kết quả hiện tại và không được trình bày như thành tích đã đạt.

GIỚI HẠN VÀ PHẠM VI DIỄN GIẢI

Một tập chia theo thời gian và một seed cho mỗi cấu hình chưa đủ để kết luận về mọi giai đoạn thị trường hoặc mọi website. Khác biệt phân phối giữa giai đoạn train và test có thể ảnh hưởng chất lượng dù chương trình đúng về số học.

Retailrocket chỉ xét các phiên còn sự kiện kế tiếp; phiên một sự kiện không có nhãn trong bài toán này. Việc loại timestamp mở hồ ảnh hưởng một phần giao dịch nhiều sản phẩm. Các chỉ số không được diễn giải thành tỷ lệ chuyển đổi của toàn bộ khách truy cập.

Bộ S&P 500 là dữ liệu lịch sử với giới hạn về thành phần chỉ số và điều chỉnh giá. Dự báo return hoặc hướng thay đổi chưa tạo thành đánh giá chiến lược giao dịch vì chưa mô hình hóa chi phí, thanh khoản và khả năng thực thi.

RNN cơ bản có cửa sổ hữu hạn, trạng thái ẩn nhỏ và khó khăn về phụ thuộc dài hạn. Nghiên cứu tập trung giải thích và kiểm chứng mô hình này; không coi việc tăng độ phức tạp là bảo đảm cải thiện.

TÀI LIỆU THAM KHẢO (1/2)

[1] Elman, J. L. Finding Structure in Time. Cognitive Science, 14, 179-211, 1990.

https://doi.org/10.1207/S15516709COG1402_1

[2] Pascanu, R.; Mikolov, T.; Bengio, Y. On the difficulty of training recurrent neural networks. ICML, 2013.

<https://proceedings.mlr.press/v28/pascanu13.html>

[3] PyTorch. RNN: định nghĩa toán học, hình dạng đầu vào và đầu ra.

<https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html>

[4] Keras. SimpleRNN: trạng thái, masking và phép hồi tiếp.

https://keras.io/2/api/layers/recurrent_layers/simple_rnn/

TÀI LIỆU THAM KHẢO (2/2)

[5] Retailrocket. Recommender system dataset. Kaggle.

<https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

[6] Nugent, C. S&P 500 stock data. Kaggle.

<https://www.kaggle.com/datasets/camnugent/sandp500>

[7] Chollet, F. Deep Learning with Python, Second Edition. Manning, 2021.

[8] Keras. Saving and serialization: trạng thái mô hình và optimizer.

https://keras.io/2/api/models/model_saving_apis/

Mã nguồn: https://github.com/namnguyen05-vn/PTHTTM_Assignment-06

Số lượng bản ghi và các tập chia được đo trực tiếp từ tệp tải. Tài liệu API giải thích khái niệm; các phiên bản thực nghiệm cụ thể được ghi trong hồ sơ môi trường.

PHỤ LỤC. THUẬT NGỮ

Thuật ngữ	Giải thích
RNN	Mạng cập nhật trạng thái bằng đầu vào và trạng thái trước
Hidden state	Vector biểu diễn lịch sử đã quan sát
Recurrence	Quy tắc truy hồi được áp dụng qua các bước
Time step	Vị trí quan sát; không luôn là khoảng thời gian cố định
Unrolling	Trải đồ thị hồi tiếp thành chuỗi bước tính
BPTT	Lan truyền ngược qua đồ thị đã trải theo thời gian
Padding / masking	Đệm tensor / bỏ qua vị trí không phải dữ liệu thật
Logits	Điểm số trước softmax
Log return	Logarit tỷ lệ giá liên tiếp
Lookback window	Cửa sổ lịch sử dùng dự báo
Data leakage	Thông tin không sẵn có đi vào học hoặc lựa chọn
Checkpoint	Trạng thái đã lưu của mô hình và quy trình tối ưu
Epoch	Một lượt duyệt toàn bộ tập train của cấu hình
Early stopping	Dừng theo tiêu chí validation đã quy định
Baseline	Phương pháp đối chứng để đánh giá giá trị bổ sung
Concept drift	Quan hệ thống kê thay đổi theo thời gian

Bảng 37. Thuật ngữ theo ngữ cảnh báo cáo.